Tech Science Press

http://doi.org/10.32604/cmc.2026.000000

ARTICLE

Verification-Guided Continual Maintenance of Digital Twin Models Under Evolving and Potentially Unreliable Monitoring Data

Jianyong Wang[1], Jie Liu[1], Yubo Dong[1], Tengyu Gai[1] and Fengbiao Zan[1,*]

[1]School of Intelligent Science and Engineering, Qinghai Minzu University, Xining 810007, China
*Corresponding Author: Fengbiao Zan. Email: zanfb@163.com

Version September 13, 2026 submitted to Comput Mater Contin

ABSTRACT: Digital twin (DT) models deployed in computing infrastructures require continual maintenance as operating conditions evolve and new monitoring data become available. However, newly accumulated monitoring data may reflect legitimate operating changes or unreliable observations, so directly updating the deployed model can improve adaptation but may also introduce harmful model-state transitions. Existing DT updating and continual-learning methods mainly address how models are adapted, leaving insufficiently controlled whether an adapted candidate should actually replace the current deployed state when maintenance evidence itself may be unreliable. To address this problem, we propose RAVM-DT, a verification-guided continual maintenance algorithm that explicitly separates candidate adaptation from deployment commitment. At each maintenance stage, RAVM-DT adapts the current model using update data, compares the resulting candidate with the deployed state using paired recent and bounded historical verification evidence, and commits the candidate only when the evidence supports the transition; otherwise, the current state is retained. Across the five primary lifecycle conditions, RAVM-DT reduces harmful-transition frequency by approximately 85.5%–91.4% and harmful-transition downside by 85.3%–94.9% relative to active continual-adaptation baselines, at a 6.8%–13.2% increase in lifecycle MSE. Compared with the no-maintenance reference, it reduces lifecycle MSE by approximately 34.9% while retaining nonzero adaptation. The selective-transition behavior remains qualitatively consistent across corruption severities, three model architectures, two prediction horizons, and an external production workload trace.

KEYWORDS: Digital twin; continual maintenance; model-state transition; verification; unreliable monitoring data; continual learning

1 Introduction

Digital twin (DT) models provide synchronized virtual representations for monitoring, prediction, and control in industrial, wireless, and networked computing systems [1–5]. After deployment, these models must remain aligned with operating conditions that evolve over time. Newly accumulated monitoring data enable maintenance, but they may reflect legitimate workload and resource changes or unreliable observations caused by measurement noise, stale data, communication faults, or manipulation. Directly updating the deployed model can therefore improve adaptation while also replacing a stable state with a degraded one. Figure 1 illustrates this maintenance setting.

Existing DT updating and synchronization methods maintain physical–virtual consistency as system conditions change [6–8], while broader network-DT studies emphasize adaptive modeling and lifecycle synchronization [9,10]. Continual-learning methods address sequential adaptation and knowledge retention,

© 2026 by the Authors. Submitted to Comput Mater Contin for under a Creative Commons Attribution 4.0 International License, which permitted unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

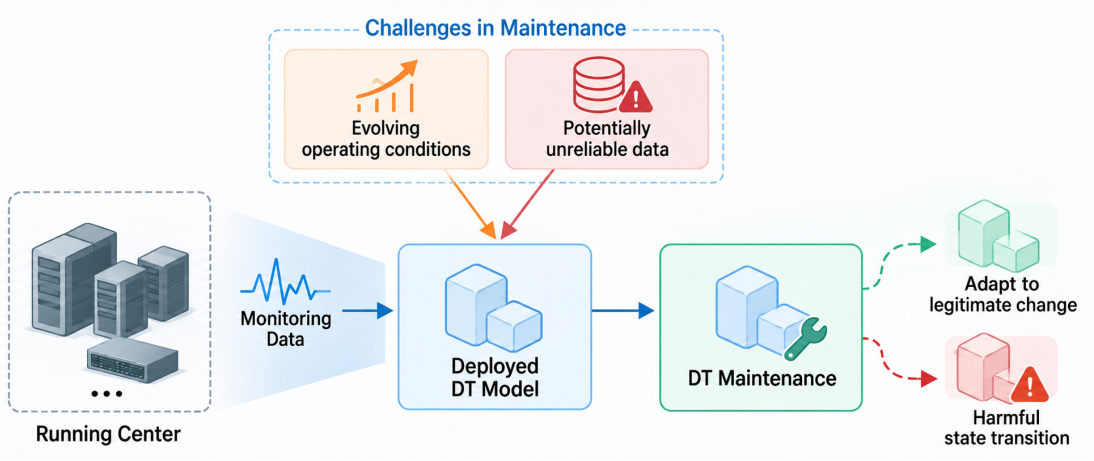


Figure 1: Problem setting for continual DT maintenance under evolving operating conditions and potentially unreliable monitoring data.

and reliability-oriented methods detect or mitigate unreliable observations before model modification. However, these lines of work do not explicitly treat post-adaptation state replacement as a separate deployment decision. When benign operating changes and unreliable observations coexist, successful adaptation on available maintenance data does not guarantee a beneficial deployed-state transition. The remaining challenge is therefore to make candidate-to-deployed state transitions reliable while retaining useful adaptation.

To address this challenge, we propose RAVM-DT, a verification-guided continual maintenance algorithm for deployed DT models. At each maintenance stage, RAVM-DT adapts the current model using update data to form a candidate, compares that candidate with the deployed state using paired recent and bounded historical verification evidence, and commits it only when both evidence criteria support replacement. Bounding the historical evidence horizon limits the influence of obsolete operating regimes while preserving temporally distinct evidence against harmful replacement.

The main contributions are as follows.

- **State-transition formulation for continual DT maintenance.** We formulate maintenance under evolving and potentially unreliable monitoring data as repeated candidate-to-deployed state transitions, characterized by lifecycle loss, harmful-transition frequency, and harmful-transition downside.
- **Verification-guided selective state transition.** RAVM-DT separates candidate adaptation from deployment commitment and uses paired recent and bounded historical evidence to decide whether to commit or retain the current state.
- **Evaluation of the safety–utility–plasticity trade-off.** We evaluate the fixed RAVM-DT configuration under benign drift, multiple corruption patterns and severities, mechanism variants, model architectures, prediction horizons, maintenance cost, and an external production workload trace.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the problem, Section 4 presents RAVM-DT, Section 5 reports the evaluation, and Section 6 concludes the paper.

2 Related Work

2.1 Digital Twin Model Updating and Synchronization

Digital-twin research has increasingly extended from one-time construction to post-deployment synchronization and updating. Li *et al.* integrated continual learning into DT-enabled edge service provisioning [6], Rodriguez *et al.* addressed data-quality control before automatic DT updating [7], and Amad *et al.* explored continual updating with in-context large-language-model adaptation [8]. Recent DT-network surveys likewise identify adaptive modeling, synchronization, and AI-assisted lifecycle management as central challenges [9,10]. These studies motivate continual maintenance, but primarily focus on how a model is updated or synchronized rather than whether an adapted candidate should replace the deployed state.

2.2 Continual Adaptation under Evolving Conditions

Continual learning studies sequential adaptation and the stability–plasticity trade-off [11–14], closely related to adaptation under concept drift [15]. Representative approaches preserve prior knowledge through parameter regularization [16], distillation [17], exemplar or episodic memory [18–20], and importance-aware constraints [21]. Recent work has also analyzed forgetting and adaptation in continual regression settings [22], while contaminated-stream methods explicitly address unreliable sequential data [23]. These methods improve adaptation, but an adapted model may still be unsuitable for deployment if the state transition itself is insufficiently supported.

2.3 Reliable and Trustworthy DT Updating

Reliable DT updating commonly emphasizes data-quality assessment, anomaly handling, and robust learning. Shen *et al.* evaluate anomalous physical-twin data before trustworthy power-system DT updates [24]. Robust estimation limits the influence of outlying observations [25], while Byzantine-robust learning and poisoning research study unreliable or adversarial training information [26,27]. These directions improve the reliability of data or learning processes, but our focus is complementary: after a candidate has been adapted, should it replace the current deployed state when recent operating changes and potentially unreliable maintenance evidence coexist? RAVM-DT therefore targets the reliability of post-adaptation model-state transitions rather than adaptation alone.

3 System Model and Problem Formulation

3.1 System Setting and Deployed DT Model

We consider a DT platform that maintains center-specific models for a set of heterogeneous computing centers $\mathcal{C}$. For center $c \in \mathcal{C}$, let a_c denote the architecture of its deployed DT model and let $\theta_{c,t}$ denote the model parameters after maintenance stage t. The deployed DT model state is represented as

$$M_{c,t} = (a_c, \theta_{c,t}), \qquad t = 0, \dots, T, \tag{1}$$

where T is the number of maintenance stages.

The initial state $M_{c,0}$ is assumed to be valid and may be obtained by any model-instantiation procedure. We focus on parameter-state maintenance after deployment and keep a_c fixed over the maintenance horizon. Repeated maintenance therefore produces the deployed-state sequence $M_{c,0} \to M_{c,1} \to \cdots \to M_{c,T}$.

3.2 Operational Data and Model-State Transition

At maintenance stage t, center c provides newly matured operational data $\mathcal{D}_{c,t}^{\text{maint}}$. The maintenance data are characterized from two complementary perspectives:

$$\mathcal{D}_{c,t}^{\text{maint}} = \mathcal{D}_{c,t}^{\text{ben}} \uplus \mathcal{D}_{c,t}^{\text{cor}}, \qquad \mathcal{D}_{c,t}^{\text{maint}} = \mathcal{D}_{c,t}^{\text{upd}} \uplus \mathcal{D}_{c,t}^{\text{ver}}, \tag{2}$$

where $\uplus$ denotes disjoint union. The first partition describes observation reliability: $\mathcal{D}_{c,t}^{\text{ben}}$ contains legitimate operating observations, whereas $\mathcal{D}_{c,t}^{\text{cor}}$ contains unreliable observations caused by measurement faults, stale data, communication faults, or manipulation. Their membership is unknown to the maintenance algorithm. The second partition specifies algorithmic roles. The update subset $\mathcal{D}_{c,t}^{\text{upd}}$ is used to generate a candidate state, whereas the chronologically later verification subset $\mathcal{D}_{c,t}^{\text{ver}}$ is excluded from candidate adaptation and provides evidence for the deployment decision. Either subset may contain unreliable observations.

For a prescribed candidate-adaptation operator $\mathcal{A}$, the candidate is

$$\widetilde{\theta}_{c,t} = \mathcal{A}\Big(\theta_{c,t-1}, \mathcal{D}_{c,t}^{\text{upd}}\Big), \qquad \widetilde{M}_{c,t} = (a_c, \widetilde{\theta}_{c,t}). \tag{3}$$

Candidate generation does not by itself modify the deployed model. Let $u_{c,t} \in \{0,1\}$ denote the deployment decision, where $u_{c,t} = 1$ commits the candidate and $u_{c,t} = 0$ retains the current state. The deployed state after maintenance is

$$M_{c,t} = \begin{cases} \widetilde{M}_{c,t}, & u_{c,t} = 1, \\ M_{c,t-1}, & u_{c,t} = 0. \end{cases} \tag{4}$$

Thus, candidate adaptation and deployment commitment are distinct stages.

3.3 Continual DT Model Maintenance Problem

Let $\mathcal{L}(\mathcal{D}; M)$ denote the mean task loss of model state M on dataset $\mathcal{D}$. Let $\mathcal{D}_{c,t}^{\text{srv}}$ denote the service interval following the stage-t deployment decision. These data are unavailable when $u_{c,t}$ is determined and are used only to evaluate the subsequent consequence. Define

$$\Delta_{c,t} = \mathcal{L}\big(\mathcal{D}_{c,t}^{\text{srv}}; \widetilde{M}_{c,t}\big) - \mathcal{L}\big(\mathcal{D}_{c,t}^{\text{srv}}; M_{c,t-1}\big). \tag{5}$$

A negative $\Delta_{c,t}$ indicates that the candidate would improve subsequent service performance relative to retaining the current state, whereas a positive value indicates degradation. A harmful state transition therefore occurs when the candidate is committed and $\Delta_{c,t} > 0$.

For the prescribed candidate-adaptation operator $\mathcal{A}$, the ideal deployment-decision problem is

$$\begin{aligned} \min_{\{u_{c,t}\}} \quad & \frac{1}{|\mathcal{C}|T} \sum_{c\in\mathcal{C}} \sum_{t=1}^{T} \mathcal{L}\big(\mathcal{D}_{c,t}^{\text{srv}}; M_{c,t}\big) \\ \text{s.t.} \quad & \frac{1}{|\mathcal{C}|T} \sum_{c\in\mathcal{C}} \sum_{t=1}^{T} u_{c,t} \mathbf{1}[\Delta_{c,t} > 0] \le \rho, \\ & \frac{1}{|\mathcal{C}|T} \sum_{c\in\mathcal{C}} \sum_{t=1}^{T} u_{c,t} [\Delta_{c,t}]_+ \le \epsilon, \end{aligned} \tag{6}$$

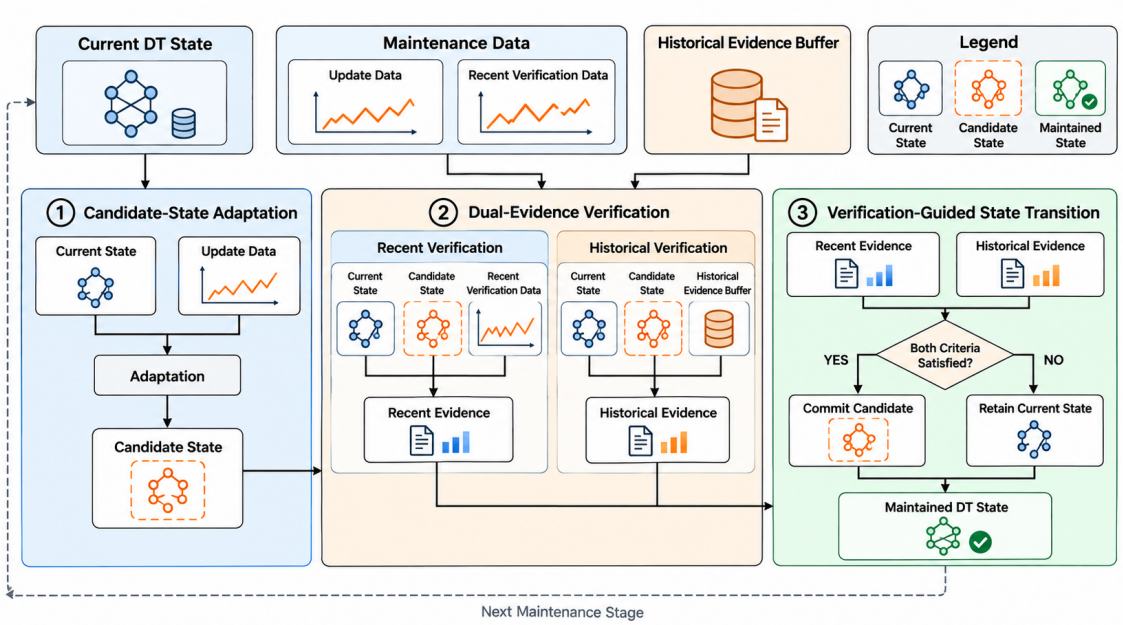


Figure 2: Overview of RAVM-DT. A single candidate state is adapted from the current DT state using update data, evaluated through recent and bounded historical evidence, and committed only when both criteria support replacement; otherwise, the current state is retained as the maintained DT state.

where $|C|$ is the number of computing centers, $\mathbf{1}[\cdot]$ is the indicator function, $[z]_+ = \max(z, 0)$, $\rho \in [0, 1]$ is the tolerated harmful-transition frequency, and $\epsilon \geq 0$ is the tolerated average harmful-transition downside.

Equation (6) defines an ideal lifecycle objective and risk criteria rather than an online guarantee. At maintenance time, $\boldsymbol{D}_{c,t}^{\mathrm{srv}}$ and hence $\Delta_{c,t}$ are unavailable. The practical problem is therefore to determine $u_{c,t}$ using only information available before the subsequent service interval, while maintaining lifecycle utility and limiting harmful deployed-state transitions.

4 Proposed RAVM-DT Algorithm

4.1 Overview

RAVM-DT solves the decision problem in Section 3 by separating candidate adaptation from deployment commitment. At stage t, update data produce one candidate state, recent and bounded historical verification evidence compare that candidate with the current deployed state, and a selective decision either commits the candidate or retains the current state. Current verification data enter the historical buffer only after the decision. Figure 2 summarizes this workflow.

4.2 Candidate-State Adaptation

At each stage, the candidate starts as an isolated copy of the current deployed state and is optimized only with update data. Let $\mathcal{B}_{c,t}^{(s)} \subseteq \mathcal{D}_{c,t}^{\mathrm{upd}}$ denote the update mini-batch used at step s. A gradient-based realization of the candidate-adaptation operator in Eq. (3) is

$$\begin{aligned} \theta_{c,t}^{(0)} &= \theta_{c,t-1}, \\ \theta_{c,t}^{(s+1)} &= \theta_{c,t}^{(s)} - \eta \nabla_\theta \mathcal{L}\Big(\mathcal{B}_{c,t}^{(s)}; (a_c, \theta_{c,t}^{(s)})\Big), \quad s = 0, \dots, S-1, \\ \widetilde{\theta}_{c,t} &= \theta_{c,t}^{(S)}, \qquad \widetilde{M}_{c,t} = (a_c, \widetilde{\theta}_{c,t}), \end{aligned} \tag{7}$$

where S is the stage-level adaptation budget and η the learning rate. The complete update subset defines the sampling pool, while $\mathcal{D}_{c,t}^{\mathrm{ver}}$ is excluded from parameter updates and the deployed state is unchanged until commitment.

4.3 Dual-Evidence Verification

Each verification collection is partitioned chronologically into B block units; target windows belong to one block, although their input contexts may overlap because of sliding-window construction. Let $\mathcal{E}_{c,t}^{\mathrm{rec}} = \mathcal{D}_{c,t}^{\mathrm{ver}}$ and let $\mathcal{E}_{c,t-1}^{\mathrm{hist}}$ denote blocks retained in a bounded historical buffer. Before the first online stage, $\mathcal{H}_{c,0}$ is initialized from target-specific pre-maintenance verification evidence. For $q \in \{\mathrm{rec}, \mathrm{hist}\}$ and chronological block $\mathcal{E}_{c,t,j}^{q}$, RAVM-DT evaluates the candidate and current state on the same block:

$$d_{c,t,j}^{q} = \mathcal{L}\big(\mathcal{E}_{c,t,j}^{q}; \widetilde{M}_{c,t}\big) - \mathcal{L}\big(\mathcal{E}_{c,t,j}^{q}; M_{c,t-1}\big). \tag{8}$$

Negative values favor the candidate. The paired differences are summarized by

$$\begin{aligned} m_{c,t}^{q} &= \mathrm{median}_j \, d_{c,t,j}^{q}, \\ r_{c,t}^{q} &= \mathrm{median}_j \Big| d_{c,t,j}^{q} - m_{c,t}^{q} \Big|, \\ U_{c,t}^{q} &= m_{c,t}^{q} + \kappa_q r_{c,t}^{q}, \qquad q \in \{\mathrm{rec}, \mathrm{hist}\}, \end{aligned} \tag{9}$$

where $\kappa_q \geq 0$ controls the penalty for disagreement among verification blocks. $U_{c,t}^{q}$ is a robust conservative evidence score, not a formal statistical confidence bound.

4.4 Verification-Guided State Transition

Let $\delta_{\mathrm{imp}} \geq 0$ be the required recent improvement and $\delta_{\mathrm{ret}} \geq 0$ the tolerated historical degradation. The deployment decision is

$$u_{c,t} = \begin{cases} 1, & U_{c,t}^{\mathrm{rec}} \leq -\delta_{\mathrm{imp}} \text{ and } U_{c,t}^{\mathrm{hist}} \leq \delta_{\mathrm{ret}}, \\ 0, & \text{otherwise.} \end{cases} \tag{10}$$

Thus, recent evidence requires adaptation benefit while historical evidence limits degradation of recently retained behavior. The historical buffer has capacity C_{hist}; after the stage-t decision, current verification blocks are appended and the oldest blocks are evicted if necessary. This ordering prevents current verification data from influencing their own decision through the historical branch and limits the effect of obsolete regimes.

Algorithm 1 (RAVM-DT Maintenance for Center c at Stage t): 1. **Input:** $M_{c,t-1}$, $\mathcal{D}_{c,t}^{\mathrm{upd}}$, $\mathcal{D}_{c,t}^{\mathrm{ver}}$, historical buffer $\mathcal{H}_{c,t-1}$, and method parameters.
2. Adapt on $\mathcal{D}_{c,t}^{\mathrm{upd}}$ using Eq. (7) to obtain $\widetilde{M}_{c,t}$.
3. Form recent and historical evidence and compute Eqs. (8)–(9).
4. Determine $u_{c,t}$ by Eq. (10) and update $M_{c,t}$ by Eq. (4).
5. **After the decision**, append current verification blocks to $\mathcal{H}_{c,t-1}$ and evict the oldest blocks if required.
6. **Return:** $M_{c,t}$ and the updated buffer $\mathcal{H}_{c,t}$.

4.5 Decision-Parameter Calibration

Let $\gamma \in \Gamma$ denote a candidate decision-parameter configuration. On calibration data disjoint from final evaluation, let $\widehat{\mathcal{R}}(\gamma)$, $\widehat{\mathcal{H}}(\gamma)$, and $\widehat{\mathcal{S}}(\gamma)$ denote empirical lifecycle loss, harmful-transition frequency, and harmful-transition downside. We select

$$\gamma^* \in \arg\min_{\gamma\in\Gamma} \widehat{\mathcal{R}}(\gamma) \quad \text{s.t.} \quad \widehat{\mathcal{H}}(\gamma) \le \rho, \qquad \widehat{\mathcal{S}}(\gamma) \le \epsilon. \tag{11}$$

The selected configuration is fixed before final evaluation. Equation (11) is an offline calibration rule; online decisions still use only information available before the subsequent service interval.

5 Experimental Evaluation

5.1 Experimental Setup

5.1.1 Benchmark and Lifecycle Protocol

Workload prediction is used as the maintenance task, following its role in proactive cloud/resource management [28–31]. Method development, decision calibration, and final evaluation use disjoint center pools; the final pool contains 30 centers, three fixed model seeds per center, and 24 chronological stages. For each target center, the initial deployed model is obtained by source pretraining followed by 50-step target warm-up adaptation on the first 800 raw observations. A disjoint 200-observation pre-maintenance interval initializes six historical verification blocks. Each stage then follows a raw-first causal split of 400 update, 240 verification, and 400 service observations; windows are formed separately within each role, giving 304/144/304 windows for $L = 96, H = 1$. The preceding service interval becomes the next stage's update interval only after its labels mature. The main predictor is a one-layer GRU-32; architecture/horizon generality is examined in Section 5.6.

5.1.2 Method Configuration and Calibration

Method structure is fixed on the development pool and decision parameters are calibrated on a disjoint pool before final evaluation. Each verification subset is divided into six chronological blocks, and the 12-block history is updated by FIFO truncation only after the current decision. The decision search uses $\kappa_{\mathrm{rec}}, \kappa_{\mathrm{hist}} \in \{0, 0.5, 1.0, 1.5\}$; among configurations satisfying the absolute harmful-transition and downside criteria on mixed, persistent-bias, and replay calibration settings, lifecycle MSE is minimized with a deterministic tie-break. Table 1 reports the fixed configuration.

5.1.3 Conditions, Baselines, Metrics, and Statistics

The four benign regimes apply a deterministic scalar workload shift: stationary remains unchanged, gradual ramps from 0 to 0.8, abrupt switches to 0.8 at the stage-9 verification boundary, and recurring alternates between shifted and unshifted intervals. The mixed condition overlays telemetry corruption with

Table 1: Main Experimental Configuration

Category	Item	Setting
Lifecycle	Centers / seeds / stages	30 / 3 / 24
Predictor	Architecture / input / horizon	GRU-32 / $L = 96$ / $H = 1$
Adaptation	Optimizer / loss / updates	SGD / MSE / 50
	Learning rate / clipping	0.01 / 1.0
	Batch / sampling / weight decay	64 / replacement / 0
Verification	Blocks per stage / history capacity	6 / 12 blocks
	$\kappa_{\mathrm{rec}}, \kappa_{\mathrm{hist}}$	0, 0.5
Commitment	$\delta_{\mathrm{imp}}, \delta_{\mathrm{ret}}$	2.8018×10^{-5}, 0
Risk criteria	ρ, ϵ	0.05, 5.6036×10^{-5}
Statistics	Unit / bootstrap	Center / 4000 repetitions
	Model / attack / bootstrap seeds	7311–7313 / 9417 / 2904
Hardware	GPU	RTX 3060 Laptop GPU

nominal fraction $\alpha = 0.10$ on each center's base regime. Each role segment is divided into ten temporal strata and $\lfloor n_s\alpha \rfloor$ positions per stratum are corrupted; mixed corruption cycles random noise, stage-persistent bias, bursts, and replay, while service data remain clean. Persistent-bias, replay, and additional α values are examined in Section 5.3.

We compare RAVM-DT with Always-FT, experience replay (ER) [20], elastic weight consolidation (EWC) [16], Huber-loss fine-tuning (Huber-FT) [25], ValidationCommit, and Frozen. Active methods share the 50-step adaptation budget; ER uses FIFO raw replay, EWC uses $\lambda = 10$ with Fisher EMA $\gamma = 0.9$, and Huber-FT uses $\delta = 0.1$. ValidationCommit uses the whole-verification mean candidate–current loss difference with a calibrated threshold; its calibration triggered the retain-all fallback, yielding zero final-pool commitments. Lifecycle MSE is the primary utility metric; transition safety is measured by harmful-transition frequency and average downside, with commit rate and beneficial-commit recall characterizing adaptation. Centers are the statistical units: seeds are averaged within center and paired 95% intervals use 4000 center-cluster bootstrap repetitions. For all methods, service data are excluded from candidate adaptation and deployment commitment.

5.2 Overall Lifecycle Maintenance Performance

Figure 3 shows the central safety–utility–plasticity trade-off. Active always-update baselines achieve lower MSE, whereas RAVM-DT incurs about 6.8%–13.2% higher MSE but reduces harmful-transition frequency by about 85.5%–91.4% and harmful downside by 85.3%–94.9%. ValidationCommit makes no final-pool commitments and therefore follows Frozen; relative to this no-update reference, RAVM-DT lowers MSE by about 34.9% while committing roughly one quarter of candidate states. Thus, its safety gain is not obtained by disabling adaptation.

5.3 Robustness to Unreliable Monitoring Data

Across the nine tested corruption–severity combinations in Fig. 4, harmful-transition frequency remains below the 5% synthetic-domain calibration criterion. Mixed corruption produces the clearest utility/downside deterioration as α increases, while commit rates decline across all three corruption types. The safety metric is not monotone in nominal corruption fraction, so the result is interpreted as robustness within the evaluated range rather than a general severity law.

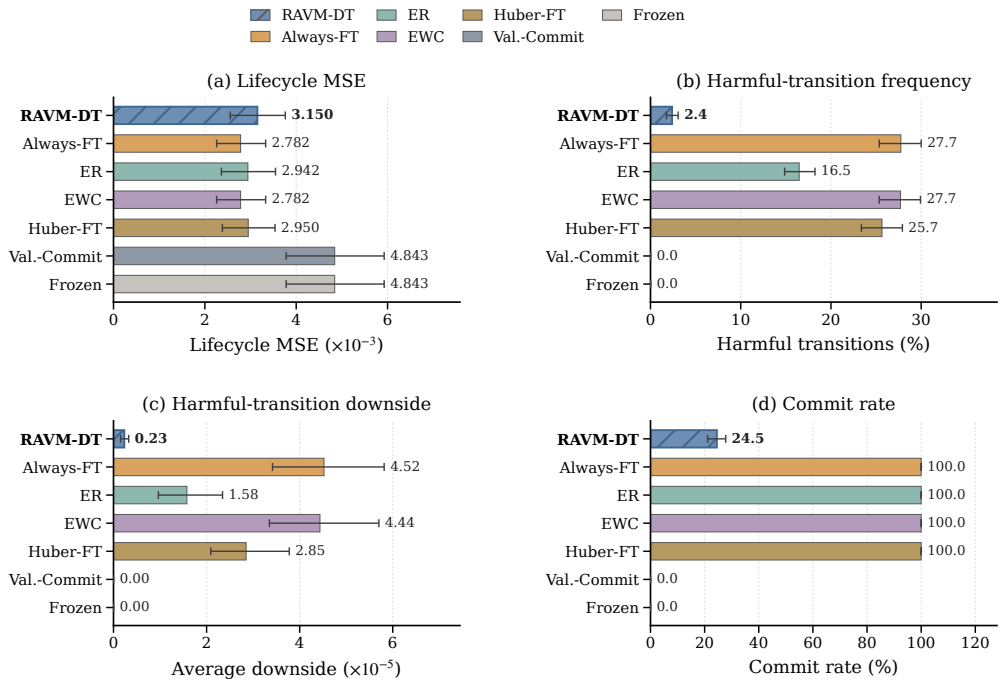


Figure 3: Overall lifecycle performance over the five primary conditions: (a) MSE, (b) harmful-transition frequency, (c) average harmful downside, and (d) commit rate. Error bars denote 95% center-cluster bootstrap intervals.

5.4 Adaptation under Benign Operating Evolution

Always-FT remains the lowest-MSE reference, but RAVM-DT substantially outperforms Frozen under gradual, abrupt, and recurring drift. Its commit rate and beneficial-commit recall are higher under gradual, abrupt, and recurring drift than under stationary conditions, while harmful-transition frequency remains about 1.0%–2.6% across the four benign regimes. The method therefore retains selective plasticity at a measurable utility cost relative to unrestricted adaptation.

5.5 Mechanism Analysis

Figure 6(a) provides no evidence that the tested candidate-side complexity variants—blockwise construction or trimmed state-delta aggregation—improve the final maintenance outcome, supporting the simpler single-path candidate. Figure 6(b) shows that recent-only verification is more plastic but reaches about 12.6% harmful transitions under mixed corruption; adding history sharply reduces this exposure, while bounding the retained history recovers utility relative to a longer history. Finally, Fig. 6(c) shows that selective commitment strongly reduces harmful replacement at the cost of retaining more candidates. Together, these results support simple candidate adaptation followed by bounded temporal verification and selective deployment control.

5.6 Sensitivity, Generality, and Maintenance Cost

The sensitivity grid in Fig. 7(a)–(b) exposes an explicit utility–risk trade-off; the fixed setting lies on the observed frontier but is not claimed to be universally optimal. Without architecture- or horizon-specific retuning, the expected qualitative ordering is retained in all 12 tested architecture–horizon–condition cells: RAVM-DT pays a positive MSE cost relative to Always-FT while reducing harmful-transition frequency. This supports consistency across the tested settings, not architecture-invariant performance.

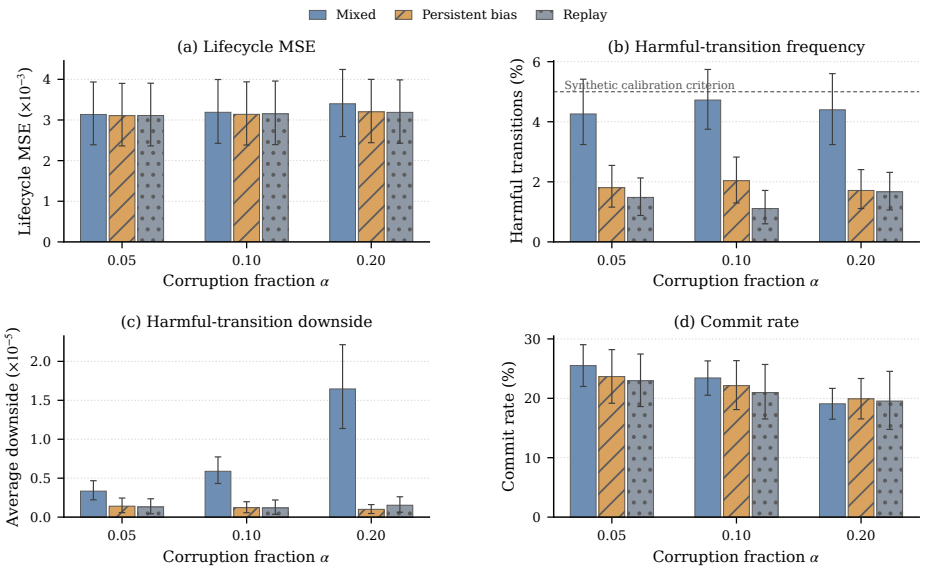


Figure 4: Robustness across corruption severities. Mixed corruption affects update and verification data; persistent bias and replay target verification evidence. Panels report MSE, harmful-transition frequency, harmful downside, and commit rate for $\alpha \in \{0.05, 0.10, 0.20\}$.

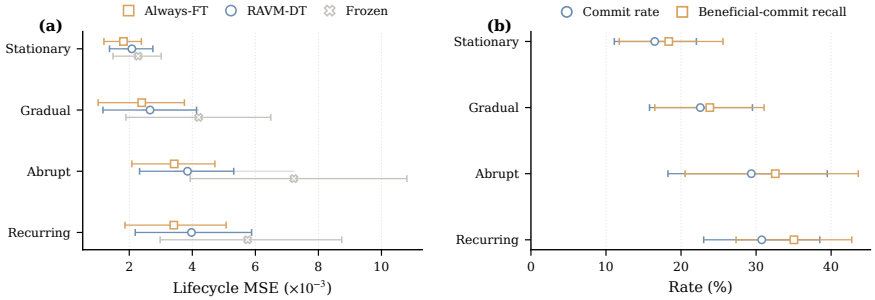


Figure 5: Benign operating evolution: (a) lifecycle MSE for RAVM-DT, Always-FT, and Frozen; (b) RAVM-DT commit rate and beneficial-commit recall. Error bars denote 95% center-cluster bootstrap intervals.

Table 2 shows that RAVM-DT requires about 0.185 s per maintenance stage, 55.7 MB peak GPU memory, and a 155.5 kB persistent buffer on the evaluated platform. Timing covers update construction, adaptation, verification, commitment, and maintenance bookkeeping, but excludes subsequent service evaluation and file I/O. Its runtime is of the same order as the adaptive baselines; these measurements characterize the implementation rather than a general latency guarantee.

5.7 External Production-Trace Evaluation

We finally evaluate the fixed configuration on the reconstructed 40-machine Alibaba Cluster Trace split [32], without target-domain recalibration. For each selected machine, the first 17,000 valid timestamp-sorted observations are retained without interpolation; seven telemetry values are represented with value/mask channels and processed by the same causal lifecycle protocol. Under clean workloads, RAVM-DT is close to the active baselines in MSE while showing much lower harmful-transition exposure. Under mixed corruption, it remains selectively active and strongly reduces harmful transitions/downside relative to active baselines, but its harmful-transition frequency reaches 6.49%, above the 5% synthetic-domain calibration criterion, and its MSE is slightly higher than Frozen. The result therefore supports qualitative transfer

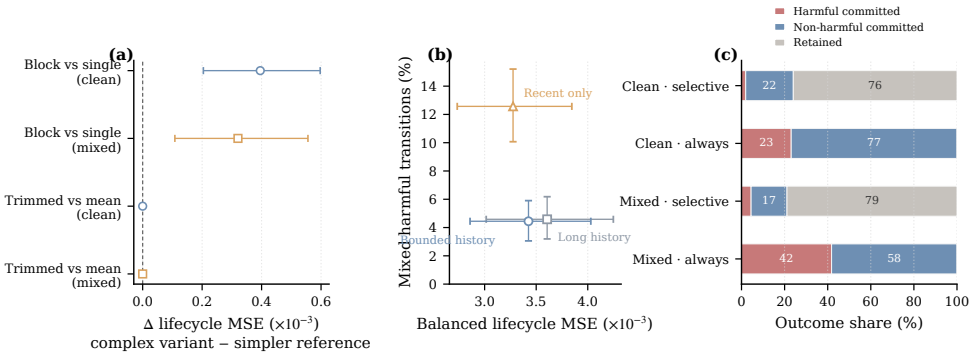


Figure 6: Mechanism analysis on the development pool: (a) paired MSE effects of additional candidate-side complexity, (b) utility–risk trade-off of recent-only, long-history, and bounded-history verification, and (c) deployment outcomes for selective and always-commit variants.

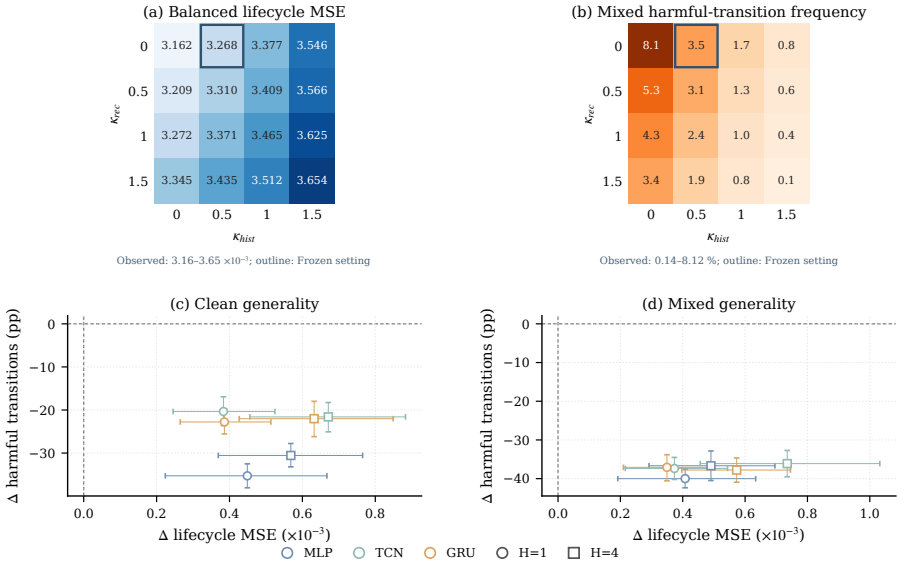


Figure 7: Sensitivity and generality of RAVM-DT. Panels (a)–(b) show the predefined $\kappa_{\text{rec}} \times \kappa_{\text{hist}}$ grid; the outlined cell is the fixed setting $(0, 0.5)$. Panels (c)–(d) show clean and mixed effects, computed as RAVM-DT minus Always-FT, across MLP, TCN, and GRU with $H \in \{1, 4\}$.

of selective state-transition control, while showing that the exact risk operating point can be domain dependent.

6 Conclusion

This paper presented RAVM-DT for continual maintenance of deployed DT models under evolving and potentially unreliable monitoring data. By separating candidate adaptation from deployment commitment, RAVM-DT uses paired recent and bounded historical verification evidence to decide whether an adapted candidate should replace the current state. Across the five primary lifecycle conditions, it reduces harmful-transition frequency by about 85.5%–91.4% and harmful downside by 85.3%–94.9% relative to active adaptation baselines, at a 6.8%–13.2% MSE increase; relative to Frozen, it lowers MSE by about 34.9% while retaining nonzero adaptation. Mechanism, benign-drift, and architecture/horizon results support the intended selective-transition behavior rather than a retain-all policy. On Alibaba workloads, the same

Table 2: Measured Maintenance Cost in the Final Benchmark

Method	Runtime/stage (s)	Peak GPU (MB)	Buffer (kB)
RAVM-DT	0.185	55.7	155.5
Always-FT	0.250	55.6	0
ER	0.262	98.3	302.4
EWC	0.504	58.3	44.8
Huber-FT	0.250	55.6	0
ValidationCommit	0.252	67.7	0
Frozen	0.004	25.5	0

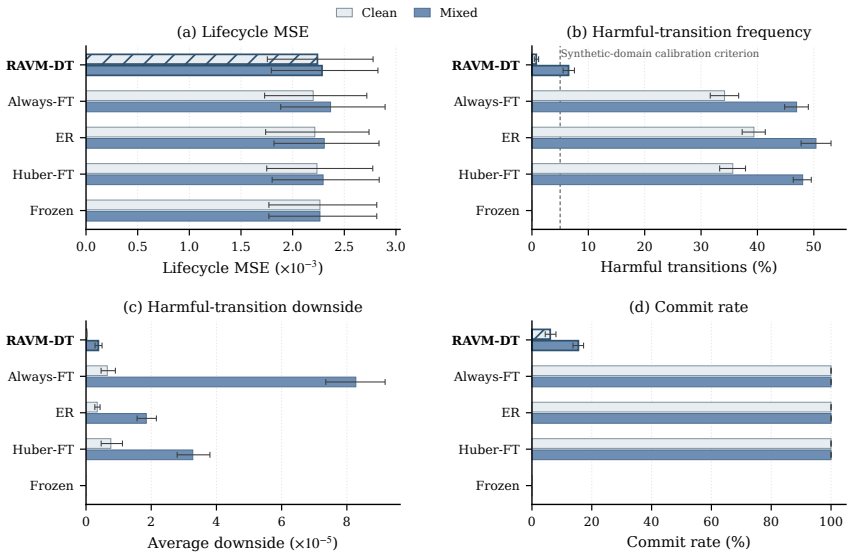


Figure 8: External Alibaba-trace evaluation under clean and mixed corruption. Panels report (a) lifecycle MSE, (b) harmful-transition frequency, (c) harmful downside, and (d) commit rate. The 5% line in (b) is the synthetic-domain calibration criterion.

qualitative safety control transfers, but the 6.49% mixed-condition harmful-transition frequency exceeds the 5% synthetic calibration criterion, indicating domain-dependent risk calibration. Current evidence is empirical and limited to workload prediction on one controlled benchmark and one external trace; future work will study calibration under domain shift, additional DT tasks/platforms, and mechanisms that improve adaptation without sacrificing transition safety.

Acknowledgement: During the preparation of this manuscript, the authors used OpenAI ChatGPT (GPT-5.6 Sol) for language polishing and editorial revision. The authors subsequently reviewed and revised the relevant content and accept full responsibility for the article.

Funding Statement: This work was supported in part by the Major Science and Technology Special Project of Qinghai Office of Science and Technology, China, under Grant No. 2024-GX-A3.

Author Contributions: The authors confirm contribution to the paper as follows: conceptualization, Jianyong Wang; methodology, Jianyong Wang; software, Jianyong Wang; validation, Jianyong Wang; formal analysis, Jianyong Wang; investigation, Jianyong Wang; data curation, Jianyong Wang; visualization, Jianyong Wang; writing—original draft preparation, Jianyong Wang; writing—review and editing, Jie Liu, Yubo Dong, Tengyu Gai and Fengbiao Zan. All authors reviewed and approved the final version of the manuscript.

Availability of Data and Materials: The implementation code, experimental configurations, and processed results supporting the findings of this study are available from the corresponding author upon reasonable request. The Alibaba Cluster Trace used for external evaluation is a publicly available third-party dataset and should be obtained from the original provider; it is not redistributed with this article.

Ethics Approval: Not applicable. This study does not involve human participants or animals.

Conflicts of Interest: The authors declare no conflicts of interest. The funder had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

1. Tao F, Zhang H, Liu A, Nee AYC. Digital Twin in Industry: State-of-the-Art. IEEE Transactions on Industrial Informatics. 2019;15(4):2405-15.
2. Khan LU, Han Z, Saad W, Hossain E, Guizani M, Hong CS. Digital Twin of Wireless Systems: Overview, Taxonomy, Challenges, and Opportunities. IEEE Communications Surveys & Tutorials. 2022;24(4):2230-54.
3. Mihai S, Yaqoob M, Hung DV, Davis W, Towakel P, Raza M, et al. Digital Twins: A Survey on Enabling Technologies, Challenges, Trends and Future Prospects. IEEE Communications Surveys & Tutorials. 2022;24(4):2255-91.
4. Xu H, Wu J, Pan Q, Guan X, Guizani M. A Survey on Digital Twin for Industrial Internet of Things: Applications, Technologies and Tools. IEEE Communications Surveys & Tutorials. 2023;25(4):2569-98.
5. Wu Y, Zhang K, Zhang Y. Digital Twin Networks: A Survey. IEEE Internet of Things Journal. 2021;8(18):13789-804.
6. Li J, Guo S, Liang W, Wang J, Chen Q, Zeng Y, et al. Digital Twin-Enabled Service Provisioning in Edge Computing via Continual Learning. IEEE Transactions on Mobile Computing. 2024;23(6):7335-50.
7. Rodriguez F, Chicaiza WD, Sanchez A, Escano JM. Updating Digital Twins: Methodology for Data Accuracy Quality Control Using Machine Learning Techniques. Computers in Industry. 2023;151:103958.
8. Amad H, Astorga N, van der Schaar M. Continuously Updating Digital Twins Using Large Language Models. In: Proceedings of the 42nd International Conference on Machine Learning. vol. 267 of Proceedings of Machine Learning Research; 2025. p. 1343-66.
9. Qin B, Pan H, Dai Y, Si X, Huang X, Yuen C, et al. Machine and Deep Learning for Digital Twin Networks: A Survey. IEEE Internet of Things Journal. 2024;11(21):34694-716.
10. Hakiri A, Gokhale A, Ben Yahia S, Mellouli N. A Comprehensive Survey on Digital Twin for Future Networks and Emerging Internet of Things Industry. Computer Networks. 2024;244:110350.
11. Parisi GI, Kemker R, Part JL, Kanan C, Wermter S. Continual Lifelong Learning with Neural Networks: A Review. Neural Networks. 2019;113:54-71.
12. De Lange M, Aljundi R, Masana M, Parisot S, Jia X, Leonardis A, et al. A Continual Learning Survey: Defying Forgetting in Classification Tasks. IEEE Transactions on Pattern Analysis and Machine Intelligence. 2022;44(7):3366-85.
13. Wang L, Zhang X, Su H, Zhu J. A Comprehensive Survey of Continual Learning: Theory, Method and Application. IEEE Transactions on Pattern Analysis and Machine Intelligence. 2024;46(8):5362-83.
14. Son J, Lee S, Kim G. When Meta-Learning Meets Online and Continual Learning: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence. 2025;47(1):413-32.
15. Gama J, Zliobaite I, Bifet A, Pechenizkiy M, Bouchachia A. A Survey on Concept Drift Adaptation. ACM Computing Surveys. 2014;46(4):1-37.
16. Kirkpatrick J, Pascanu R, Rabinowitz N, Veness J, Desjardins G, Rusu AA, et al. Overcoming Catastrophic Forgetting in Neural Networks. Proceedings of the National Academy of Sciences. 2017;114(13):3521-6.
17. Li Z, Hoiem D. Learning without Forgetting. In: European Conference on Computer Vision; 2016. p. 614-29.
18. Rebuffi SA, Kolesnikov A, Sperl G, Lampert CH. iCaRL: Incremental Classifier and Representation Learning. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2017. p. 2001-10.
19. Lopez-Paz D, Ranzato M. Gradient Episodic Memory for Continual Learning. In: Advances in Neural Information Processing Systems. vol. 30; 2017. p. 6467-76.
20. Rolnick D, Ahuja A, Schwarz J, Lillicrap T, Wayne G. Experience Replay for Continual Learning. In: Advances in Neural Information Processing Systems. vol. 32; 2019. p. 348-58.
21. Aljundi R, Babiloni F, Elhoseiny M, Rohrbach M, Tuytelaars T. Memory Aware Synapses: Learning What (not) to Forget. In: European Conference on Computer Vision; 2018. p. 139-54.

22. Ding M, Ji K, Wang D, Xu J. Understanding Forgetting in Continual Learning with Linear Regression. In: Proceedings of the 41st International Conference on Machine Learning. vol. 235 of Proceedings of Machine Learning Research. PMLR; 2024. p. 10978-1001.
23. Bang J, Koh H, Park S, Song H, Ha JW, Choi J. Online Continual Learning on a Contaminated Data Stream With Blurry Task Boundaries. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition; 2022. p. 9275-84.
24. Shen Z, Sun P, Arrano-Vargas F, Konstantinou G. Anomalous Data Evaluation for Reliable Updates of Trustworthy Power System Digital Twins. Expert Systems with Applications. 2026;314:131434.
25. Huber PJ. Robust Estimation of a Location Parameter. The Annals of Mathematical Statistics. 1964;35(1):73-101.
26. Yin D, Chen Y, Kannan R, Bartlett P. Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates. In: Proceedings of the 35th International Conference on Machine Learning. vol. 80 of Proceedings of Machine Learning Research; 2018. p. 5650-9.
27. Biggio B, Nelson B, Laskov P. Poisoning Attacks against Support Vector Machines. In: Proceedings of the 29th International Conference on Machine Learning; 2012. p. 190.
28. Kim YM, Song S, Koo BM, Son J, Lee Y, Baek JG. Enhancing Long-Term Cloud Workload Forecasting Framework: Anomaly Handling and Ensemble Learning in Multivariate Time Series. IEEE Transactions on Cloud Computing. 2024;12(2):789-99.
29. Zuo Z, Huang Y, Li Z, Jiang Y, Liu C. Mixed Contrastive Transfer Learning for Few-Shot Workload Prediction in the Cloud. Computing. 2025;107(1):5.
30. Zhao F, Lin W, Lin S, Tang S, Li K. MSCNet: Multi-Scale Network with Convolutions for Long-Term Cloud Workload Prediction. IEEE Transactions on Services Computing. 2025;18(2):969-82.
31. Feng B, Ding Z. Application-Oriented Cloud Workload Prediction: A Survey and New Perspectives. Tsinghua Science and Technology. 2025;30(1):34-54.
32. Group A. Alibaba Cluster Trace Program: Cluster Trace v2018. Alibaba Group; 2018. GitHub repository; accessed 2026-07-03.